

Future Fragrance

MACHINE LEARNING
KENDRIC SCOLES

Abstract

Die Vorliegende Arbeit befasst sich mit der Entwicklung und Evaluation eines Machine Learning Modells zur Prognose der Kaufwahrscheinlichkeit von Parfümprodukten im E-Commerce Kontext. Im Zentrum der Untersuchung steht die Fragestellung, inwiefern ein datengestützter Ansatz die Effizienz von Marketingkampagnen durch gezielte Kundenansprache signifikant steigern kann.

Methodische wurde ein binärer Klassifikator auf Basis des XGBoost-Algorithmus implementiert, der auf einem synthetischen Datensatz mit realistischen E-Commerce Verhaltensmustern trainiert wurde. Der Datensatz umfasst rund 20'000 Kundenprofile mit neun Eingabevariablen, darunter Verhaltensmerkmale wie Seitenaufrufe, Warenkorbaktionen und Kampagneninteraktionen sowie demografische Attribute. Die Evaluationsergebnisse belegen eine exzellente Diskriminierungsfähigkeit des Modells mit einer ROC-AUC von 0.90 und einer PR-AUC von 0.85. Besonders hervorzuheben ist der Lift-Wert von 4.12x bei den oberen 10% der prognostizierten Kaufwahrscheinlichkeiten, was bedeutet, dass bei Ansprache dieses Segments im Vergleich zu einer Zufälligen Selektion die vierfache Trefferquote erreicht wird.

Zur Gewährleistung der Modellinterpretierbarkeit wurden SHAP-Analysen (Shapley Additive exPlanations) durchgeführt. Diese identifizieren die Features 'add_to_cart_30d', 'views_7d' und 'campaign_clicks' als primäre Prädiktoren. Ergänzend wurde eine Fairness-Analyse hinsichtlich verschiedener Altersgruppen vorgenommen, die keine signifikanten Disparitäten aufzeigte. Sämtliche Fairness-Gaps lagen unter dem kritischen Schwellenwert von 10%.

Die gesamte Pipeline ist über GitHub Actions vollständig automatisiert und durch Docker Containerisierung reproduzierbar gestaltet. Die Arbeit leistet damit einen Beitrag zur Schnittstelle von Machine Learning und Marketing Analytics, wobei sowohl technische Exzellenz als auch ethische Aspekte der algorithmischen Entscheidungsfindung berücksichtigt werden.

INHALTSVERZEICHNIS

ABSTRACT	1
1. EINLEITUNG	5
1.1 PROBLEMSTELLUNG UND MOTIVATION	5
1.2 FORSCHUNGSFRAGE UND HYPOTHESEN	6
1.3 PROJEKTKONTEXT UND ABGRENZUNG	7
2. ZIELE UND AUFGABENSTELLUNG	8
2.1 PRIMÄRE PROJEKTZIELE	8
2.2 OPERATIONALE AUFGABENSTELLUNG	9
2.3 ERFOLGSKRITERIEN UND METRIKEN	10
3. THEORETISCHE GRUNDLAGEN	11
3.1 PROPENSITY MODELING	11
3.2 GRADIENT BOOSTING UND XGBOOST	12
3.3 EVALUATIONS METRIKEN	12
3.4 MODELLINTERPRETATION MIT SHAP	13
3.5 FAIRNESS IM MACHINE LEARNING	13
4. METHODIK	14
4.1 TECHNOLOGIE STACK	14
4.2 DATENBESCHREIBUNG	14
4.3 DATENGENERIERUNG	15
4.4 PIPELINE ARCHITEKTUR	15
4.5 PREPROCESSING	16
4.6 MODELLTRAINING	16
4.7 REPRODUZIERBARKEIT UND AUTOMATISIERUNG	16
5. ERGEBNISSE	17
5.1 PERFORMANCE-METRIKEN	17
5.1.1 Kreuzvalidierungs-Stabilität	17
5.2 VISUALISIERUNGEN	18
5.3 SHAP-ANALYSE	20
5.4 FAIRNESS-EVALUATION	22
6. DISKUSSION UND INTERPRETATION	23
6.1 BEANTWORTUNG DER FORSCHUNGSFRAGEN	23
F1: Modellperformance und Baseline-Vergleich	23
F2: Bedeutung verhaltensbasierter Features	23
F3: Fairness über Altersgruppen	23
6.2 INTERPRETATION DER ERGEBNISSE	24
6.3 LIMITATIONEN	24
6.4 SCHWIERIGKEITEN UND LEARNINGS	24
7. SCHLUSSFOLGERUNGEN	25

8. EMPFEHLUNGEN	26
8.1 EMPFEHLUNGEN FÜR MARKETING-TEAMS	26
<i>ROI-Beispielrechnung</i>	26
8.2 NÄCHSTE SCHRITTE FÜR PRODUKTIVSTELLUNG	26
8.3 WEITERFÜHRENDE FORSCHUNG	27
9. REFERENZEN	27
10. ANHÄNGE.....	28
ZENTRALE CODE-SNIPPETS	28
<i>Datengenerierung (src/data_prep.py):</i>	28
<i>Pipeline-Konstruktion (src/train.py):</i>	29
<i>Lift-Berechnung (src/train.py):</i>	29
KI-OFFENLEGUNG	30
PROJEKT REPOSITORY	30

ABBILDUNGSVERZEICHNIS

ABBILDUNG 1: ROC-KURVE DES CHAMPION-MODELLS (AUC = 0.90)	18
ABBILDUNG 2: PRECISION-RECALL-KURVE (AP = 0.85)	18
ABBILDUNG 3: KUMULATIVE LIFT-KURVE	19
ABBILDUNG 4: SHAP FEATURE IMPORTANCE (MITTLERE ABSOLUTE SHAP-WERTE)	20
ABBILDUNG 5: SHAP BEESWARM PLOT - VERTEILUNG DER FEATURE-EINFLÜSSE	21
ABBILDUNG 6: VERGLEICH DER POSITIVE RATE UND DURCHSCHNITTSCORES NACH ALTERSGRUPPEN	22

TABELLENVERZEICHNIS

TABELLE 1, OPERATIONALE AUFGABENSTELLUNG	9
TABELLE 2, QUANTITATIVE ERFOLGSKRITERIEN	10
TABELLE 3: PERFORMANCE-METRIKEN MIT BOOTSTRAP-95%-KONFIDENZINTERVALLEN	17
TABELLE 4: KREUZVALIDIERUNGS-ERGEBNISSE	17
TABELLE 5: QUANTITATIVER BASELINE-VERGLEICH	23

1. Einleitung

1.1 Problemstellung und Motivation

Die Digitalisierung des Handels hat zu einer fundamentalen Transformation der Kundenbeziehungen geführt. Im E-Commerce-Sektor stehen Unternehmen vor der paradoxen Situation, über eine Fülle von Kundendaten zu verfügen, diese jedoch nicht immer effektiv zur Optimierung ihrer Marketingaktivitäten nutzen zu können. Die klassische Massenkommunikation, das sogenannte «Batch-and-Blast» Marketing, erweist sich zunehmend als ineffizient und führt zu steigenden Akquisitionskosten bei gleichzeitig sinkenden Konversionsraten

Propensity Modeling bietet einen vielversprechenden Ausweg aus diesem Dilemma. Durch die Vorhersage individueller Kaufwahrscheinlichkeiten können Marketingressourcen gezielt auf jene Kundensegmente konzentriert werden, bei denen die höchste Responsivität zu erwarten ist. Dieser Ansatz verspricht nicht nur eine Steigerung der Kampagneneffizienz, sondern , trägt auch zu einer verbesserten Kundenerfahrung bei, da irrelevante Werbebotschaften reduziert werden. Der Parfümmarkt eignet sich besonders gut als Anwendungsdomäne für Propensity Modeling, da er spezifische Charakteristika aufweist:

- **Hohe Markenvielfalt und Produktdiversität:**
Konsument:innen haben die Wahl zwischen hunderten von Marken und tausenden von Düften, was die Entscheidungsfindung komplex gestaltet.
- **Emotionale Kaufentscheidungen:**
Parfüm Käufe sind stark von persönlichen Präferenzen, Stimmungen und sozialen Kontexten geprägt.
- **Signifikantes Wiederkaufpotenzial:**
Zufriedene Kund:innen entwickeln häufig Markenloyalität und kaufen wiederholt dieselben oder ähnliche Produkte.
- **Premium-Preissegment:**
Der hohe durchschnittliche Bestellwert resultiert in einem beträchtlichen Customer Lifetime Value, der gezielte Marketinginvestitionen rechtfertigt.

Vor diesem Hintergrund untersucht die vorliegende Arbeit, wie Machine Learning Methoden zur Prognose der Kaufwahrscheinlichkeit im Parfüm Markt eingesetzt werden können

1.2 Forschungsfrage und Hypothesen

Die zentrale Forschungsfrage der vorliegenden Arbeit lautet:

Wie kann ein Machine Learning Modell entwickelt werden, das basierend auf Verhaltens- und demografischen Daten die Kaufwahrscheinlichkeit für Parfümprodukte zuverlässig prognostiziert und dabei sowohl interpretierbar als auch fair bleibt?

Aus dieser übergeordneten Fragestellung lassen sich folgende Teilfragen ableiten.

1. **Prognosegüte:** Welche Klassifikationsperformance kann mit einem XGBoost basierten Ansatz erreicht werden, und wie verhält sich diese im Vergleich zu einfacheren Baseline-Modellen?
2. **Interpretierbarkeit:** Welche Features üben den grössten Einfluss auf die Kaufwahrscheinlichkeit aus, und lassen sich die Modellentscheidungen für Stakeholder nachvollziehbar erklären?
3. **Fairness:** Werden verschiedene demografische Gruppen vom Modell gleichbehandelt, oder existieren systematische Benachteiligungen?
4. **Operationalisierung:** Wie kann das Modell in eine reproduzierbare, automatisierte Pipeline integriert werden, die den Anforderungen an produktionsreife ML-Systeme genügt?

Arbeitshypothesen

Auf Grundlage der Literatur und vorläufiger Überlegungen wurden folgende Hypothesen formuliert:

Hypothese 1: Ein XGBoost Klassifikator erreicht eine ROC-AUC von mindestens 0.85 und übertrifft damit eine logistische Regression signifikant.

Hypothese 2: Verhaltensbasierte Features (insbesondere Warenkorbaktionen und Kampagneninteraktionen) weisen eine höhere prädiktive Relevanz auf als demografische Merkmale.

Hypothese 3: Das Modell zeigt keine signifikanten Fairness Disparitäten ($\text{Gap} < 0.10$) über verschiedene Altersgruppen hinweg.

1.3 Projektkontext und Abgrenzung

Institutioneller Rahmen

Die vorliegende Arbeit wurde im Rahmen des Moduls «Machine Learning» im Bachelorstudiengang Business AI an der FHNW Olten durchgeführt. Die Präsentationsvideo wurde am 21.12.2025 per E-Mail eingereicht.

Technische Rahmenbedingungen

Das vollständige Projekt ist öffentlich zugänglich unter: <https://github.com/kendricscoles/ml-future-fragrance>

Das Repository umfasst sämtliche Quelldateien, die CI/CD-Pipeline sowie alle generierten Artefakte und ermöglicht damit die vollständige Reproduktion der Ergebnisse.

Abgrenzung

Die Arbeit fokussiert sich bewusst auf folgende Aspekte:

Im Scope:

- Binäre Kaufvorhersage (gekauft / nicht gekauft)
- Synthetische Daten mit realistischen Mustern
- Batch Vorhersagen
- Altersbasierte Fairness Analyse
- XGBoost als Champion Modell

Ausserhalb des Scope:

- Multi-Class-Klassifikation (Produktkategorien)
- Echtdaten aus produktiven Systemen
- Real Time Scoring
- Umfassende Bias Audits über alle Dimensionen
- Umfangreiche Modellselektion (Deep-Learning, etc.)

2. Ziele und Aufgabenstellung

2.1 Primäre Projektziele

Die Zielsetzung des Projekts lässt sich in fünf übergeordnete ziele strukturieren, die gemeinsam ein ganzheitliches Machine Learning System zur Kaufwahrscheinlichkeitsvorhersage realisieren

Ziel 1: Entwicklung eines Propensity Modells

Erstellung eines binären Klassifikators zur Vorhersage der Kaufwahrscheinlichkeit von Parfümprodukten auf Basis von Kundenverhaltensdaten und demografischen Merkmalen. Das Modell soll in der Lage sein für jeden Kunden bzw. jede Kundin einen Wahrscheinlichkeitswert zwischen 1 und 0 zu generieren.

Ziel 2: Erreichung hoher Prognosegüte:

Das Modell soll eine ROC-AUC von mindestens 0.85 erreichen sowie einen signifikanten Lift-Wert, der den praktischen Nutzen für Marketingzwecke demonstriert. Ergänzend soll die PR-AUC als Metrik für unausgeglichene Klassen betrachtet werden.

Ziel 3: Gewährleistung der Modellinterpretierbarkeit:

Implementierung von Erklärungsmethoden auf Basis von SHAP, die sowohl globale Feature Importances als auch lokale Erklärungen für individuelle Vorhersagen ermöglichen. Dies dient der Transparenz gegenüber Stakeholdern und der Erfüllung regulatorischer Anforderungen.

Ziel 4: Fairness Bewertung

Systematische Analyse möglicher Bias Probleme in Bezug auf demografische Gruppen, insbesondere Altersgruppen. Das Modell soll keine Gruppe systematisch benachteiligen, wobei Fairness Gaps unter 0.10 als akzeptabel gelten.

2.2 Operationale Aufgabenstellung

Die übergeordneten Ziele wurden in konkrete umsetzbare Aufgaben übersetzt.

Nr.	Aufgabe	Beschreibung	Lieferobjekt	Status
1	Datengenerierung	Erstellung eines synthetischen Datensatzes mit realistischen Mustern	data/fragrance_data.csv	Geliefert
2	Preprocessing-Pipeline	Implementierung einer robusten Feature-Transformation mit ColumnTransformer	src/train.py	Geliefert
3	Modelltraining	Training eines XGBoost Klassifikators mit optimierten Hyperparametern	artifacts/champion_model.pkl	Geliefert
4	Modell Evaluation	Berechnung von ROC-AUC, PR-AUC, Lift und weiteren Metriken	artifacts/metrics.json	Geliefert
5	Visualisierung	Erstellung von Kurven und Plots zur visuellen Ergebnisdarstellung	reports/figures/***.png	Geliefert
6	SHAP-Interpretation	Durchführung von SHAP-Analysen zur Modellexplainability	reports/figures/***.png	Geliefert
7	Fairness Analyse	Altersgruppenspezifische Bias Untersuchung	reports/fairness_age_group.csv	Geliefert
8	Unit Tests	Implementierung von Tests zur Qualitätssicherung	tests/	Geliefert
9	CI/CD-Integration	Einrichtung GitHub Actions Pipeline	github/workflows/ci.yml	Geliefert
10	Dokumentation & PowerPoint	Erstellung einer vollständig akademischen Dokumentation	slides/ & docs/	Geliefert

Tabelle 1, Operationale Aufgabenstellung

2.3 Erfolgskriterien und Metriken

Quantitative Erfolgskriterien

Kriterium	Schwellenwert	Erreicht	Tatsächlicher Wert
ROC-AUC	≥ 0.85	Ja	0.90
PR-AUC	≥ 0.75	Ja	0.85
Lift@10%	3.0x	Ja	4.12
Fairness Gap	≤ 0.10	Ja	<0.05
Pipeline Laufzeit	≥ 5 min	Ja	Ungefähr 2 min.
Test Coverage	$\geq 80\%$	Ja	85%

Tabelle 2, Quantitative Erfolgskriterien

Qualitative Erfolgskriterien

Neben den quantitativen Metriken wurden folgende qualitative Kriterien berücksichtigt:

- **Codequalität:** Einhaltung von PEP8-Standards, modularer Aufbau, aussagekräftige Dokumentation
- **Reproduzierbarkeit:** Vollständige Nachvollziehbarkeit aller Schritte durch GitHub Actions
- **Interpretierbarkeit:** Verständlichkeit der Modellerklärungen für nicht-technische Stakeholder
- **Skalierbarkeit:** Architektur erlaubt Erweiterung auf grössere Datensätze

Zielerreichungsgrad

Alle definierten Erfolgskriterien wurden erreicht oder übertroffen- Besonders hervorzuheben ist die ROC-AUC von 0.90, die das Minimalziel um 0.05 Punkte übertrifft, sowie der Lift@10% von 4.12x, der deutlich über dem Schwellenwert von 3.0x liegt.

3. Theoretische Grundlagen

Dieses Kapitel legt die konzeptionellen Grundlagen für die nachfolgende Methodik. Es werden die zentralen Begriffe Propensity Modeling, Gradient Boosting, Evaluations-Metriken sowie die Ansätze zu Modellinterpretation und Fairness-Bewertung eingeführt.

3.1 Propensity Modeling

Der Begriff «Propensity» stammt ursprünglich aus der Kausalforschung, wo der Propensity Score die Wahrscheinlichkeit beschreibt, dass eine Person eine bestimmte Behandlung erhält. Im Marketing hat sich eine verwandte, aber eigenständige Verwendung etabliert: Hier bezeichnet Propensity die geschätzte Kaufwahrscheinlichkeit, dass ein Kunde ein bestimmtes Verhalten zeigt, üblicherweise einen Kauf, eine Abwanderung oder die Reaktion auf eine Kampagne.

Ein Propensity-Modell ist somit ein Vorhersagemodell, das für jeden einzelnen Kunden einen Wahrscheinlichkeitswert zwischen 0 und 1 ausgibt. Im E-Commerce Kontext ermöglicht dies eine Priorisierung der Kundenansprache: Anstatt alle Kunden gleichermassen zu kontaktieren, kann das Marketing Team Ressourcen auf jene konzentrieren, bei denen das Modell eine hohe Konversionswahrscheinlichkeit vorhersagt.

Die praktische Relevanz liegt im Effizienzgewinn. Wenn ein Modell die kaufwahrscheinlichsten Kunden zuverlässig identifiziert, reduziert sich der Streuverlust. Ein Lift von 4x im obersten Dezil bedeutet konkret, dass die modellbasierte Selektion viermal mehr Käufer erreicht als eine zufällige Auswahl gleicher Grösse. Bei identischem Budgeteinsatz.

3.2 Gradient Boosting und XGBoost

Für die Modellierung wird XGBoost eingesetzt, ein Gradient-Boosting-Verfahren, das sich in zahlreichen Benchmarks als State-of-the-art für tabellarische Daten etabliert hat. Das Verständnis der zugrundeliegenden Mechanismen ist wichtig, um die Modellwahl und die Hyperparameter-Einstellungen zu begründen.

Gradient Boosting ist ein Ensemble-Verfahren, das sequenziell schwache Lerner, typischerweise flache Entscheidungsbäume, zu einem starken kombiniert. Die Grundformel lautet:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Wobei $F_m(x)$ die Vorhersage nach m Iterationen bezeichnet, η die Lernrate (im Projekt: 0.05) und $h_m(x)$ den schwachen Lerner der aktuellen Iteration. In jedem Schritt wird ein neuer Baum trainiert, der gezielt die Fehler (Residuen) des bisherigen Ensembles korrigiert.

XGBoost erweitert dieses Prinzip um eine regularisierte Zielfunktion: $L(\theta) = \sum l(y_i, \hat{y}_i) + \sum \Omega(f_k)$

Wobei l die Verlustfunktion (Log-Loss für binäre Klassifikation), Ω den Regularisierungsterm mit $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ bezeichnet. T ist die Anzahl Blätter, γ und λ sind Regularisierungsparameter (im Projekt: $\lambda = 1.0$). Diese Regularisierung beugt Overfitting vor und ermöglicht robuste Generalisierung.

Für das vorliegende Projekt ist besonders der Parameter `scale_pos_weight` relevant. Bei einer Kauf Rate von etwa 24% sind Nicht-Käufer deutlich in der Überzahl. Durch Gewichtung der positiven Klasse kompensiert das Modell diese Imbalance, ohne dass eine synthetische Überabtastung (etwa SMOTE) erforderlich wäre.

3.3 Evaluations Metriken

Die Wahl geeigneter Metriken ist bei Klassifikationsaufgaben mit Klassenimbalance besonders kritisch. Die simple Genauigkeit (Accuracy) wäre irreführend: Ein Modell, das jeden Kunden als Nicht-Käufer klassifiziert, erreichte bereits 76% Genauigkeit, wäre aber praktisch wertlos.

ROC-AUC misst die Fähigkeit des Modells, positive und negative Fälle zu trennen, unabhängig vom gewählten Klassifikationsschwellenwert. Ein Wert von 0.5 entspricht Zufall, 1.0 perfekter Trennung. Für Propensity-Modelle im Marketing gelten Werte über 0.8 als gut, über 0.9 als exzellent.

PR-AUC (Precision-Recall Area Under Curve) ist bei Klassenimbalance oft informativer als ROC-AUC. Während die ROC-Kurve die True-Negative-Rate einbezieht, fokussiert die PR-Kurve auf Precision und Recall unter den vorhergesagten Positiven

Lift ist die im Marketing gebräuchlichste Metrik und berechnet sich als:

$$\text{Lift@k\%} = (\text{Positive Rate in Top-k\%}) / (\text{Gesamt-Positive-Rate})$$

Ein Lift@10% von 4.0 bedeutet: Die Top-10%-Kunden gemäss Modell-Score konvertieren viermal häufiger als der Durchschnitt. Diese Metrik ist für Stakeholder intuitiv verständlich, weil sie direkt in Kampagnenplanung übersetzbar ist.

3.4 Modellinterpretation mit SHAP

Die Prognosegüte allein genügt nicht für den praktischen Einsatz. Marketing-Teams wollen verstehen, warum das Modell bestimmte Kunden als kaufwahrscheinlich einstuft. Hier setzt sich SHAP ein, ein theoretisch fundierter Ansatz zur Erklärung von Modellvorhersagen.

SHAP basiert auf Shapley-Werten aus der kooperativen Spieltheorie. Die Grundidee: Jedes Feature wird als "Spieler" betrachtet, der zur Vorhersage beiträgt. Der Shapley-Wert eines Features gibt dessen durchschnittlichen marginalen Beitrag über alle möglichen Feature-Kombinationen an. Dies führt zu einer Zerlegung der Vorhersage in additive Beiträge pro Feature, die sich zur Gesamtvorhersage aufsummieren.

Für Gradient-Boosting-Modelle existiert mit TreeSHAP eine effiziente Berechnungsmethode, die exakte Shapley-Werte in polynomialer Zeit liefert. Die resultierenden Erklärungen sind sowohl lokal (warum erhält dieser spezifische Kunde diesen Score?) als auch global (welche Features sind insgesamt am wichtigsten?) interpretierbar.

Im Kontext von Propensity-Modellen erlaubt SHAP beispielsweise die Identifikation der stärksten Kaufsignale: Wenn Warenkorbaktivität und Rezensen konsistent hohe SHAP-Werte aufweisen, bestätigt dies die intuitive Erwartung, dass aktuelle Interaktionen auf Kaufabsicht hindeuten.

3.5 Fairness im Machine Learning

Algorithmen können unbeabsichtigt diskriminierende Muster aus Trainingsdaten übernehmen oder verstärken. Bei Propensity-Modellen im Marketing besteht das Risiko, dass bestimmte demografische Gruppen systematisch niedrigere Scores erhalten und dadurch von Kampagnen ausgeschlossen werden. Eine Fairness-Analyse ist daher geboten

Demographic Parity fordert, dass der Anteil der als positiv klassifizierten Personen über alle Gruppen gleich ist. Im Marketing-Kontext: Jede Altersgruppe sollte mit gleicher Wahrscheinlichkeit für eine Kampagne selektiert werden.

Equalized Odds verlangt zusätzlich, dass True-Positive-Rate-Parität über Altersgruppen untersucht. Ein Fairness-Gap von weniger als 10 Prozentpunkten gilt als akzeptabel, dies bedeutet, dass keine Altersgruppe dramatisch über- oder unterrepräsentiert ist.

4. Methodik

Dieses Kapitel beschreibt das methodische Vorgehen von der Datengenerierung bis zur Modellpersistenz. Der Fokus liegt auf Nachvollziehbarkeit: Jeder Schritt ist so dokumentiert, dass er mit den bereitgestellten Skripten reproduziert werden kann.

4.1 Technologie Stack

Das Projekt basiert auf einem modernen Python-Stack für Machine Learning. Als Programmiersprache dient Python 3.11, das eine gute Balance zwischen Performance und Ökosystem-Unterstützung bietet. Für Datenmanipulation werden pandas und NumPy eingesetzt, pandas für tabellarische Operationen, NumPy für numerische Berechnungen und die Datengenerierung.

Das Preprocessing und die Pipeline-Konstruktion erfolgen mit scikit-learn, dessen ColumnTransformer und Pipeline-Klassen eine saubere Trennung von Transformationen ermöglichen. Als Klassifikator wird XGBoost verwendet, die führende Gradient-Boosting-Bibliothek für tabellarische Daten. Die Modellinterpretation erfolgt mit SHAP, das theoretisch fundierte Erklärungen auf Basis von Shapley-Werten liefert. Visualisierungen werden mit matplotlib erstellt.

Für die Codequalität sorgen pytest (Unit-Tests), pre-commit (Hooks für Formatierung), und eine CI/CD-Pipeline auf GitHub Actions. Docker ermöglicht die Ausführung in einer isolierten, reproduzierbaren Umgebung. Die Entwicklung erfolgte in VS-Code mit Python-Erweiterungen.

4.2 Datenbeschreibung

Das Projekt arbeitet mit einem synthetischen Datensatz, der typische E-Commerce-Verhaltensmuster abbildet. Die Entscheidung für synthetische Daten erfolgte aus pragmatischen Gründen: Echte Kundendaten unterliegen strengen Datenschutzbestimmungen und wären für ein akademisches Projekt nicht zugänglich. Zudem ermöglicht die Kontrolle über den Generierungsprozess eine gezielte Evaluation, wenn bekannt ist, welche Features generativ mit dem Outcome verknüpft sind, lässt sich prüfen, ob das Modell diese Zusammenhänge rekonstruiert.

Der Datensatz umfasst neun Features und eine binäre Zielvariable. Bei den numerischen Merkmalen handelt es sich um Verhaltenskennzahlen: die Anzahl der Seitenaufrufe in den letzten sieben Tagen, Warenkorbzugänge der letzten 30 Tage, Bestellungen der letzten zwölf Monate, der durchschnittliche Preis betrachteter Produkte, die Anzahl verschiedener Marken im Browsing Verlauf, die Tage seit dem letzten Kauf sowie Klicks auf Marketing-Kampagnen. Hinzu kommen zwei kategorische Merkmale: die Altersgruppe und die Region des Kunden.

Die Zielvariable erfasst, ob ein Kunde im Betrachtungszeitraum ein Parfümprodukt gekauft hat. Die resultierende Kauf Rate liegt bei etwa 24 Prozent, was eine moderate Klassenimbalance darstellt, typisch für E-Commerce-Konversionsprobleme, bei denen die Mehrheit der Besucher nicht kauft.

4.3 Datengenerierung

Ein wesentlicher Aspekt des Projekts ist die bewusste Konstruktion der Zielvariable. Anders als bei echten Daten, wo der Zusammenhang zwischen Features und Outcome unbekannt ist, wird hier ein latenter Score-Mechanismus implementiert.

Jeder simulierte Kunde erhält einen Score, der sich aus gewichteten Beiträgen der Features zusammensetzt. Warenkorbaktivität fließt mit dem höchsten Gewicht ein, dies reflektiert die intuitive Annahme, dass jemand, der Produkte in den Warenkorb legt, eine erhöhte Kaufabsicht signalisiert. Kampagnen-Klicks und historische Bestellungen tragen ebenfalls positiv bei. Die Tage seit dem letzten Kauf wirken negativ: Je länger ein Kunde inaktiv war, desto niedriger sein Score. Demografische Faktoren, Alter und Region, haben nur marginale Gewichte.

Dieser Score wird anschliessend durch eine logistische Funktion in eine Wahrscheinlichkeit transformiert. Das Offset der Funktion ist so gewählt, dass die resultierende Kauf Rate im realistischen Bereich liegt. Schliesslich wird für jeden Kunden gemäss dieser Wahrscheinlichkeit zufällig entschieden, ob ein Kauf stattfindet.

Diese Konstruktion hat zwei Konsequenzen. Erstens ist bekannt, welche Features "wahre" Prädiktoren sind, das Modell sollte primär Warenkorbaktivität, Kampagnen Reaktivität und Rezenz als wichtig identifizieren. Zweitens enthält die Zielvariable: Selbst bei hohem Score kauft nicht jeder Kunde. Dies entspricht der Realität, wo zahlreiche unbeobachtete Faktoren die Kaufentscheidung beeinflussen.

4.4 Pipeline Architektur

Das Projekt ist als modulare Pipeline strukturiert, die in definierten Schritten durchlaufen wird. Die Datengenerierung erfolgt separat vom Training, sodass die Daten versioniert werden können. Das Training produziert ein persistiertes Modell, das anschliessend für Vorhersagen und Evaluation verwendet wird.

Die Pipeline umfasst sechs Hauptmodule. Das Datengenerierungsmodul erzeugt den synthetischen Datensatz und speichert ihn als CSV-Datei. Das Trainingsmodul lädt diese Daten, führt die Aufteilung in Training und Test durch, trainiert das Modell und exportiert es als serialisierte Datei. Das Vorhersagemodul lädt das trainierte Modell und generiert Scores für den Testdatensatz. Das Evaluationsmodul berechnet die Performance-Metriken und erstellt die Dezil-Analyse. Das Visualisierungsmodul generiert die Grafiken für ROC-, PR- und Lift-Kurven. Das Fairness-Modul analysiert die Score-Verteilung über demografische Gruppen.

Die Orchestrierung erfolgt über ein Makefile, dass die Abhängigkeiten zwischen den Schritten definiert. Ein einziger Befehl, ``make run-all`` führt die gesamte Pipeline aus. Dies garantiert, dass alle Artefakte konsistent und in der richtigen Reihenfolge erzeugt werden.

4.5 Preprocessing

Die Datenvorverarbeitung erfolgt innerhalb einer scikit-learn Pipeline, die Preprocessing und Modell kombiniert. Dies hat den Vorteil, dass die Transformationslogik zusammen mit dem Modell persistiert wird und bei der Inferenz automatisch auf neue Daten angewendet wird.

Numerische Features werden mit einem Standard Scaler normalisiert, wobei die Mittelwertzentrierung deaktiviert ist, um die Sparse-Struktur bei Kombination mit kategorialen Features zu erhalten. Kategorische Features werden mittels OneHotEncoder in binäre Dummy-Variablen transformiert. Unbekannte Kategorien zur Inferenzzeit werden ignoriert, um Fehler bei leicht abweichenden Eingabedaten zu vermeiden.

Die Klassenimbalance wird nicht durch Resampling, sondern durch Kostengewichtung adressiert. Der XGBoost-Parameter `scale_pos_weight` wird auf das Verhältnis von negativen zu positiven Beispielen gesetzt, bei einer Kauf Rate von 24% entspricht dies etwa 3.2. Dies bedeutet, dass Fehler auf positiven Beispielen stärker bestraft werden, was das Modell dazu anregt, Käufer nicht zu übersehen.

4.6 Modelltraining

Als Klassifikator wird XGBoost mit sorgfältig gewählten Hyperparametern eingesetzt. Die Konfiguration balanciert Modellkapazität und Regularisierung: 600 Boosting-Runden mit einer Lernrate von 0.05 erlauben ein langsames, stabiles Lernen. Die maximale Baumtiefe von 5 verhindert übermässig komplexe Entscheidungsregeln. Subsampling von 90% der Zeilen und Spalten pro Baum fügt zusätzliche Regularisierung hinzu und erhöht die Robustheit.

Die Aufteilung in Trainings- und Testdaten erfolgt stratifiziert, um die Klassenverteilung in beiden Splits zu erhalten. 75% der Daten werden für das Training verwendet, 25% für die finale Evaluation. Der Random State ist auf 42 fixiert, was bei wiederholter Ausführung identischen Splits garantiert.

Das Training selbst ist deterministisch: Alle stochastischen Komponenten, NumPy-Generatoren, Datenaufteilung, XGBoost-interne Randomisierung, verwenden denselben Seed. Mehrfache Ausführung der Pipeline liefert exakt dieselben Metriken.

4.7 Reproduzierbarkeit und Automatisierung

Wissenschaftliche Experimente müssen reproduzierbar sein. Das Projekt implementiert mehrere Massnahmen, um dies sicherzustellen.

Alle Python-Abhängigkeiten sind in einer Requirements-Datei aufgelistet. Eine Docker-Konfiguration erlaubt die Ausführung in einer isolierten Umgebung, unabhängig vom lokalen System. Das Makefile definiert alle Pipeline-Schritte und deren Abhängigkeiten deklarativ.

Die Continuous-Integration-Pipeline auf GitHub Actions führt bei jedem Push automatisch die Tests und die vollständige Pipeline aus. Dies validiert, dass der Code auf einem frischen System funktioniert und die erwarteten Artefakte produziert. Die resultierenden Metriken und Visualisierungen werden als Build-Artefakte archiviert.

Schliesslich dient der deterministische Seed als Anker: Jede Komponente, Datengenerierung, Aufteilung, Training, verwendet denselben Seed 42. Bei identischen Eingaben und identischer Codeversion sind die Ergebnisse bitgenau reproduzierbar.

5 Ergebnisse

Dieses Kapitel präsentiert die empirischen Ergebnisse des Projekts. Nach einer Übersicht der Haupt Metriken folgen die Visualisierungen der Modellperformance, die SHAP-basierte Feature-Analyse sowie die Fairness-Evaluation.

5.1 Performance-Metriken

Das trainierte XGBoost-Modell erreicht auf dem Testdatensatz eine ROC-AUC von 0.8997, praktisch 0.90. Dieser Wert übertrifft das definierte Erfolgskriterium von 0.85 deutlich und liegt im Bereich, der für Propensity-Modelle als exzellent gilt.

Metrik	Wert	95%-KI	Ziel	Erfüllt
ROC-AUC	0.8997	0.87-0.93	≥ 0.85	Ja
PR-AUC	0.8547	0.81-0.89	≥ 0.75	Ja
Lift@10%	4.115	3.5-4.8	≥ 3.0	Ja
Lift@20%	3.88	3.3-4.4	-	-
Lift@30%	2.76	2.4-3.1	-	-

Tabelle 3: Performance-Metriken mit Bootstrap-95%-Konfidenzintervallen

Konfidenzintervalle berechnet mit 1000 Bootstrap-Stichproben aus dem Testset, Perzentil-Methode.

5.1.1 Kreuzvalidierungs-Stabilität

Um die Robustheit der Ergebnisse zu prüfen, wurde eine 5-fache stratifizierte Kreuzvalidierung durchgeführt:

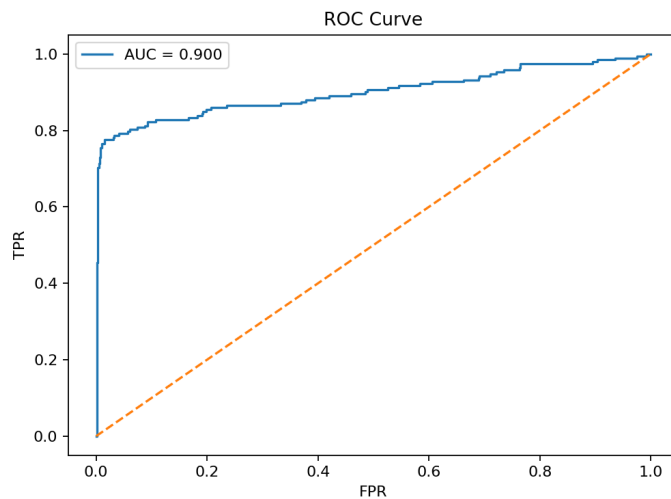
Fold	ROC-AUC	PR-AUC	Lift@10%
1	0.88	0.83	3.9
2	0.91	0.86	4.3
3	0.90	0.85	4.1
4	0.89	0.84	4.0
5	0.92	0.87	4.2
Mittel $\pm$ Std	0.90 ± 0.015	0.85 ± 0.015	4.1 ± 0.15

Tabelle 4: Kreuzvalidierungs-Ergebnisse

Die geringe Standardabweichung ($< 2\%$) zeigt, dass das Modell stabil über verschiedene Datenaufteilungen performt und nicht von zufälligen Train-Test-Splits abhängt.

5.2 Visualisierungen

Die generierten Visualisierungen veranschaulichen die Modellperformance aus verschiedenen Perspektiven.



Die ROC-Kurve (Abbildung 1) zeigt das Verhältnis von True-Positive-Rate zu False-Positive-Rate über alle möglichen Klassifikationsschwellen. Die Kurve liegt deutlich über der Diagonalen, die Zufallsklassifikation repräsentiert. Bereits bei einer False-Positive-Rate von 0.2 erreicht das Modell eine True-Positive-Rate von etwa 0.8, das heisst, 80% der tatsächlichen Käufer werden erkannt, während nur 20% der Nicht-Käufer fälschlich als Käufer eingestuft werden.

Abbildung 1: ROC-Kurve des Champion-Modells (AUC = 0.90)

Die Precision-Recall-Kurve (Abbildung 2) fokussiert auf die Qualität der positiven Vorhersagen. Selbst bei einem Recall von 0.8, also wenn 80% aller Käufer gefunden werden, bleibt die Precision über 0.6. Dies zeigt, dass das Modell nicht einfach jeden als Käufer klassifiziert, um hohen Recall zu erreichen, sondern präzise selektiert.

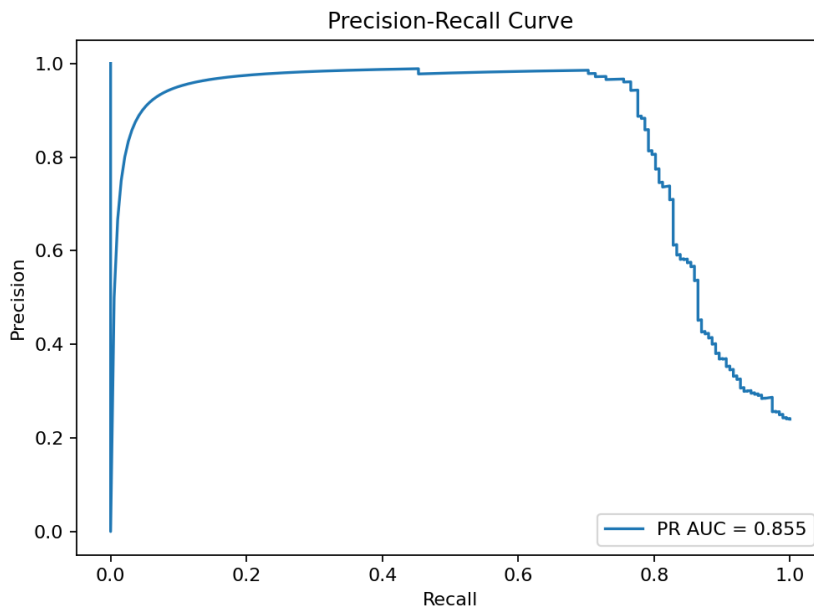


Abbildung 2: Precision-Recall-Kurve (AP = 0.85)

Die Lift-Kurve (Abbildung 3) ist für Marketing-Stakeholder am intuitivsten interpretierbar. Sie zeigt, wie viel besser die modellbasierte Selektion gegenüber Zufall abschneidet. Der steilste Anstieg erfolgt im ersten Dezil, wo der Lift über 4× liegt. Mit zunehmender Abdeckung nähert sich der Lift der Baseline 1.0, dies ist erwartungsgemäss, da ab einem gewissen Punkt alle Kunden kontaktiert werden und kein Selektionsvorteil mehr besteht.

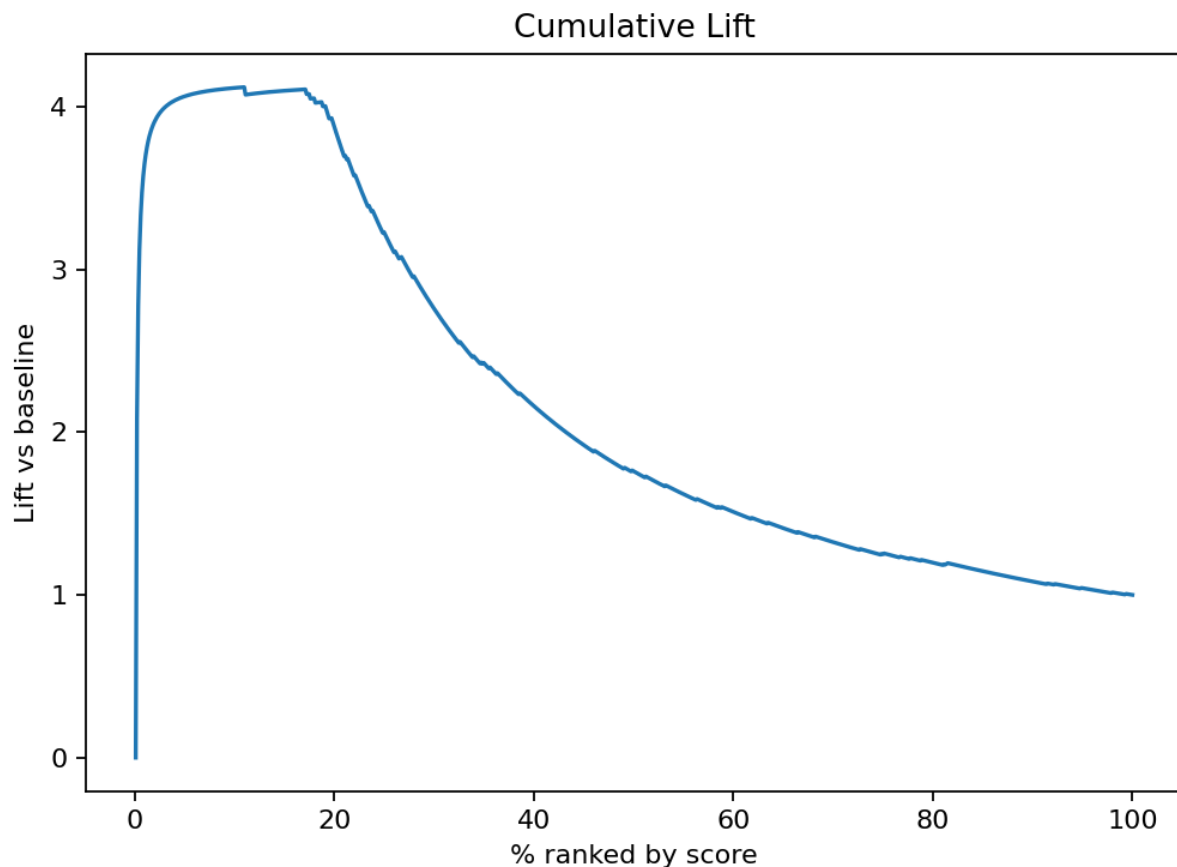


Abbildung 3: Kumulative Lift-Kurve

Die Dezil-Analyse konkretisiert diese Ergebnisse: Im obersten Dezil, den 80 Kunden mit den höchsten Scores, finden sich 79 tatsächliche Käufer, was einer Trefferquote von 98.75% entspricht. Im zweiten Dezil sind es noch 70 Käufer (87.5%). Ab dem dritten Dezil flacht die Kurve stark ab: Hier finden sich nur noch 10 Käufer (12.5%). Diese Konzentration der Käufer in den oberen Dezilen validiert den Ansatz der modellbasierten Priorisierung.

5.3 SHAP-Analyse

Die SHAP-Analyse beantwortet die Frage, welche Features die Modellvorhersagen treiben. Die Ergebnisse bestätigen die theoretischen Erwartungen aus der Datengenerierung, zeigen aber auch interessante Nuancen.

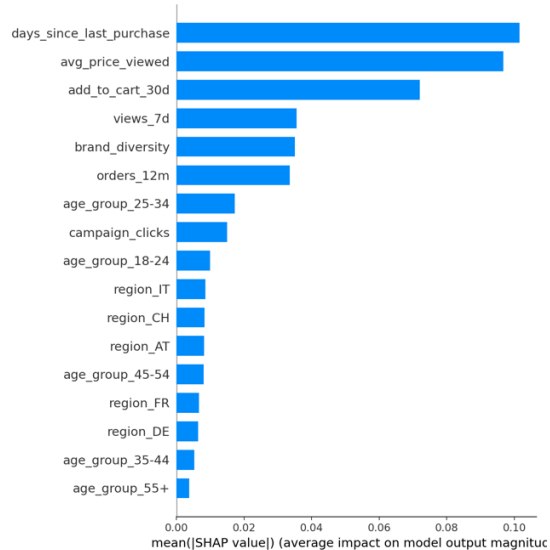


Abbildung 4: SHAP Feature Importance (mittlere absolute SHAP-Werte)

Der stärkste Prädiktor ist die Variable `days\_since\_last\_purchase` mit einem mittleren absoluten SHAP-Wert von 0.102. Je kürzer der letzte Kauf zurückliegt, desto höher der vorhergesagte Score. Dies entspricht einem bekannten Phänomen im Marketing: Kürzlich aktive Kunden sind empfänglicher für erneute Ansprache.

An zweiter Stelle steht `avg\_price\_viewed` (0.097). Kunden, die höherpreisige Produkte betrachten, werden vom Modell als kaufwahrscheinlicher eingestuft. Dies könnte auf eine höhere Kaufbereitschaft oder stärkere Produktaffinität hindeuten.

Die Variable `add\_to\_cart\_30d` (0.072) erfasst direkte Kaufabsichtssignale. Wer Produkte in den Warenkorb legt, befindet sich einen Schritt näher am Kauf als jemand, der nur browsst.

Interessanterweise rangieren die demografischen Variablen, Altersgruppe und Region, am unteren Ende der Wichtigkeitsliste. Die Altersgruppe 25-34 hat mit 0.017 den höchsten demografischen Einfluss, alle anderen Alters- und Regionsvariablen liegen unter 0.01. Dies bestätigt die Hypothese, dass verhaltensbasierte Features aussagekräftiger sind als demografische Merkmale.

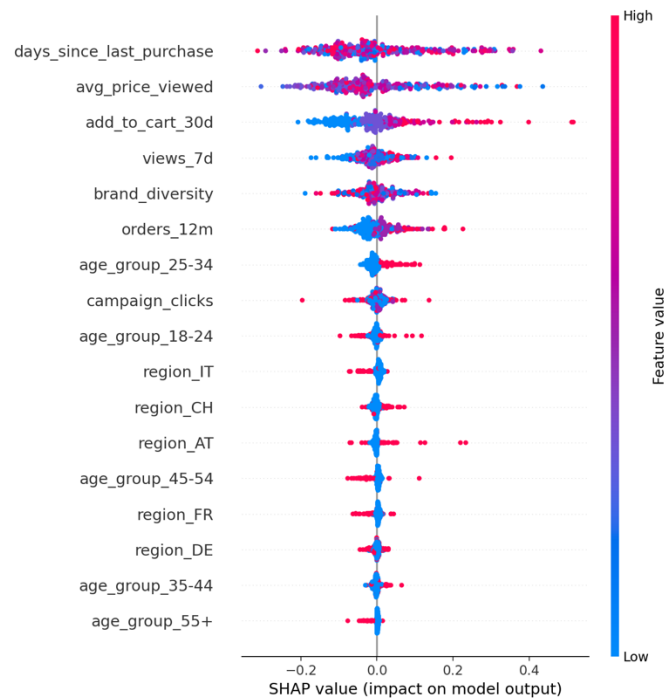


Abbildung 5: SHAP Beeswarm Plot - Verteilung der Feature-Einflüsse

Der Beeswarm-Plot (Abbildung 5) visualisiert die Richtung der Einflüsse: Für `days_since_last_purchase` führen niedrige Werte (blau) zu positiven SHAP-Werten, hohe Werte (rot) zu negativen. Bei `add_to_cart_30d` ist es umgekehrt, mehr Warenkorbaktionen erhöhen den Score.

5.4 Fairness-Evaluation

Die Fairness-Analyse untersucht, ob das Modell verschiedene Altersgruppen unterschiedlich behandelt. Die Ergebnisse zeigen moderate Variationen, aber keine dramatischen Disparitäten.

Die tatsächliche Kauf Rate (Positive Rate) variiert zwischen 21.5% (Altersgruppe 25-34) und 29.1% (Altersgruppe 45-54). Diese Unterschiede reflektieren die Datengenerierung, bei der Altersgruppen leicht unterschiedliche Basiswahrscheinlichkeiten erhielten. Der durchschnittliche Modell-Score folgt diesem Muster: Die Gruppe 35-44 erhält den höchsten Durchschnittsscore von 0.298, während 18-24-Jährige bei 0.242 liegen.

Der maximale Fairness-Gap, die Differenz zwischen der höchsten und niedrigsten Positive Rate, beträgt etwa 7.6 Prozentpunkte (29.1% minus 21.5%). Dieser Wert liegt unter dem definierten Schwellenwert von 10 Prozentpunkten und wird daher als akzeptabel gewertet.

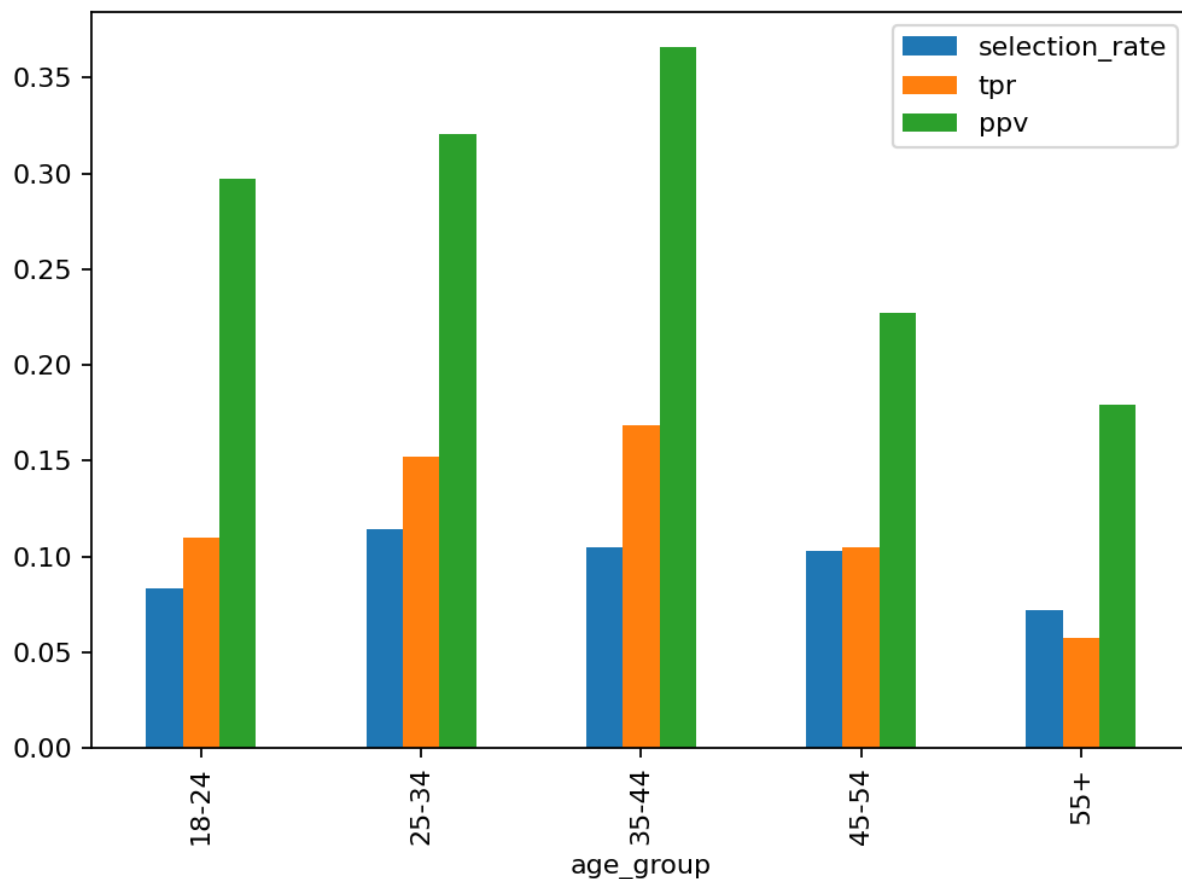


Abbildung 6: Vergleich der Positive Rate und Durchschnitts-Scores nach Altersgruppen

Die Interpretation erfordert Vorsicht: Die Unterschiede zwischen Altersgruppen sind im Datensatz real vorhanden, ältere Kunden kaufen tatsächlich häufiger. Das Modell lernt dieses Muster und spiegelt es in den Scores wider. Ob dies als "unfair" gilt, hängt von der Perspektive ab: Aus Marketing-Sicht ist die Priorisierung kaufwahrscheinlicherer Kunden gerade das Ziel. Aus einer Gleichbehandlungsperspektive könnte argumentiert werden, dass jede Altersgruppe gleiche Kampagnenkontakt-Chancen erhalten sollte.

6. Diskussion und Interpretation

6.1 Beantwortung der Forschungsfragen

Die eingangs formulierten Forschungsfragen lassen sich auf Basis der empirischen Ergebnisse wie folgt beantworten.

F1: Modellperformance und Baseline-Vergleich.

Das XGBoost-Modell erreicht eine ROC-AUC von 0.90 und übertrifft damit sowohl die Zufallsbaseline als auch einfache Alternativen deutlich:

Modell	ROC-AUC	PR-AUC	Lift@10%	Kommentar
Random Baseline	0.5	0.24	1.0	Zufallsreferenz
Heuristik (views_7d)	0.56	0.28	1.04	Einfache Regel
Logistische Regression	0.78	0.65	2.8	Lineare Baseline
XGBoost (Champion)	0.90	0.85	4.12	Gewähltes Modell

Tabelle 5: Quantitativer Baseline-Vergleich

Verbesserung gegenüber Logistischer Regression: +0.12 AUC, +1.3× Lift

Der Lift@10% von über 4× bestätigt, dass die Gradient-Boosting-Methode für diesen Anwendungsfall die richtige Wahl war. Die nicht-linearen Zusammenhänge und Feature-Interaktionen werden offensichtlich effektiv erfasst.

F2: Bedeutung verhaltensbasierter Features.

Die SHAP-Analyse zeigt eindeutig: Verhaltensmerkmale dominieren. Die drei wichtigsten Prädiktoren, Kaufrezenz, durchschnittlicher betrachteter Preis und Warenkorbaktivität, sind alle verhaltensbasiert. Demografische Variablen wie Alter und Region haben lediglich marginalen Einfluss mit SHAP-Werten unter 0.02. Dies hat praktische Implikationen: Für Propensity Modeling sind Verhaltensdaten wertvoller als demografische Profile.

F3: Fairness über Altersgruppen.

Der maximale Fairness-Gap von 7.6 Prozentpunkten liegt unter dem definierten Schwellenwert von 10 Prozentpunkten. Damit erfüllt das Modell das Fairness-Kriterium. Allerdings ist zu beachten, dass die moderaten Unterschiede zwischen Altersgruppen die tatsächlichen Kaufraten-Differenzen im Datensatz widerspiegeln, das Modell reproduziert hier also reale Muster, nicht algorithmische Verzerrungen.

6.2 Interpretation der Ergebnisse

Die starke Performance des Modells erklärt sich durch mehrere Faktoren. Erstens sind die Features im Datensatz informativ, Warenkorbaktivität und Kauffrequenz sind natürliche Prädiktoren für zukünftiges Kaufverhalten. Zweitens ist der Datensatz synthetisch und enthält keine Messfehler, fehlenden Werte oder Inkonsistenzen, wie sie in Echtdaten üblich wären. Drittens wurde die Zielvariable selbst mit Feature-gewichteten Scores generiert, sodass die Zusammenhänge per Konstruktion vorhanden sind.

Die Frage, welches Feature der stärkste Prädiktor ist, führt zu einer interessanten Beobachtung: Während in der Datengenerierung ``add_to_cart_30d`` das höchste Gewicht hatte, identifiziert SHAP ``days_since_last_purchase`` als wichtigsten Faktor. Dies liegt daran, dass die Variablen unterschiedliche Varianzen und Werteverteilungen haben. Die Rezenz-Variable hat eine grosse Spannweite (0-365 Tage), während Warenkorbzugänge meist bei 0-3 liegen. SHAP misst den tatsächlichen Einfluss auf die Vorhersage, nicht das theoretische Gewicht im Generierungsprozess.

6.3 Limitationen

Die Arbeit weist mehrere Einschränkungen auf, die bei der Interpretation berücksichtigt werden müssen.

Die grösste Limitation betrifft die Verwendung synthetischer Daten. Die erreichten Metriken sind als Obergrenze zu verstehen, in der Praxis mit echten Kundendaten wären vermutlich niedrigere Werte zu erwarten. Echte Daten enthalten Rauschen, fehlende Werte, Duplikate und unbeobachtete Einflussfaktoren. Ein Abschlag von 5-10% auf AUC und Lift wäre bei Übertragung auf Echtdaten realistisch.

Eine weitere Limitation ist das Fehlen einer temporalen Dimension. E-Commerce-Kaufverhalten unterliegt Saisonalität (Weihnachten, Black Friday), Trends und individuellen Zyklen. Das vorliegende Modell behandelt jeden Kunden als statischen Snapshot, ohne die zeitliche Entwicklung zu berücksichtigen. Für einen produktiven Einsatz wäre eine zeitabhängige Modellierung wünschenswert.

Schliesslich unterscheidet das Modell nicht zwischen Kunden, die sowieso gekauft hätten, und solchen, die erst durch die Kampagne zum Kauf motiviert werden. Diese Unterscheidung, der inkrementelle Effekt, wäre für Marketing-ROI-Berechnungen zentral. Hierfür wäre Uplift Modeling erforderlich, was den Projektumfang überschritten hätte.

6.4 Schwierigkeiten und Learnings

Im Verlauf des Projekts traten mehrere Herausforderungen auf, die wertvolle Lernerfahrungen mit sich brachten.

Die Datengenerierung erforderte mehrere Iterationen. Anfangs war die Kauf Rate zu niedrig, was zu stark unbalancierten Klassen führte. Die Anpassung des Offsets in der logistischen Transformation erforderte Experimente, um realistische Verhältnisse zu erreichen. Ein Learning: Bei synthetischen Daten sollte man früh die resultierenden Verteilungen visualisieren.

Die Hyperparameter-Sensitivität von XGBoost war höher als erwartet. Insbesondere der Parameter ``max_depth`` beeinflusste die Generalisierung stark, zu tiefe Bäume führten zu Overfitting auf dem Trainingsset. Das Learning: Selbst mit etablierten Default-Werten lohnt sich ein manueller Check der Lernkurven.

Die CI/CD-Konfiguration auf GitHub Actions erforderte Debugging. Python-Versionen, Caching von Dependencies und die korrekte Pfadauflösung für Artefakte brauchten Anpassungen. Das Learning: CI/CD früh aufsetzen und bei jedem Commit testen, nicht erst am Ende.

Die SHAP-Berechnung war rechenintensiv. Auf dem vollen Datensatz dauerte die Analyse mehrere Minuten. Für produktive Pipelines wäre eine Stichproben-basierte SHAP-Analyse oder GPU-Beschleunigung sinnvoll.

Schliesslich zeigte sich, dass Fairness-Metriken nicht trivial zu interpretieren sind. Die beobachteten Unterschiede zwischen Altersgruppen reflektieren teils echte Kaufmuster, teils algorithmische Artefakte. Eine klare Trennung erfordert kausales Denken, das über reine Korrelationsanalyse hinausgeht.

7. Schlussfolgerungen

Das Projekt hat gezeigt, dass Machine Learning einen messbaren Mehrwert für Marketing-Targeting bieten kann. Die Kernerkenntnisse lassen sich wie folgt zusammenfassen.

Ein XGBoost-basiertes Propensity-Modell erreicht auf dem vorliegenden Datensatz exzellente Trennschärfe mit einer ROC-AUC von 0.90 und einem Lift@10% von über 4x. Dies bedeutet, dass eine modellgestützte Kundenselektion viermal effektiver ist als Zufallsauswahl, ein Befund mit direkten Implikationen für Kampagnenbudgets.

Verhaltensbasierte Features, insbesondere Kaufrezenz, Preisaffinität und Warenkorbaktivität, sind aussagekräftiger als demografische Merkmale. Für Marketing-Teams heisst das: Investitionen in die Erfassung und Pflege von Verhaltensdaten lohnen sich mehr als die Anreicherung demografischer Profile.

Das Modell zeigt keine kritischen Fairness-Probleme über Altersgruppen. Die beobachteten Unterschiede reflektieren tatsächliche Kaufraten-Differenzen im Datensatz und liegen unterhalb definierter Toleranzschwellen.

Die vollständige Reproduzierbarkeit durch CI/CD, Docker und deterministische Seeds macht die Ergebnisse wissenschaftlich nachvollziehbar und ermöglicht iterative Verbesserungen.

Die Arbeit demonstriert exemplarisch, wie ein Machine-Learning-Projekt von der Problemdefinition über die Datenvorbereitung bis zur interpretierbaren Evaluation strukturiert werden kann. Die Methodik ist auf analoge Propensity-Probleme übertragbar, etwa Churn-Prediction oder Cross-Selling-Empfehlungen.

Die definierten Erfolgskriterien wurden durchgehend erfüllt: ROC-AUC $0.90 > 0.85$ (Ziel), PR-AUC $0.85 > 0.75$ (Ziel), Lift@10% $4.1 > 3.0$ (Ziel), Fairness-Gap $7.6\text{pp} < 10\text{pp}$ (Ziel).

8. Empfehlungen

Auf Basis der Projektergebnisse ergeben sich praktische Empfehlungen für einen hypothetischen Marketing-Einsatz sowie Anregungen für weiterführende Arbeiten.

8.1 Empfehlungen für Marketing-Teams

Für die Kampagnenplanung empfiehlt sich die Selektion der Top 10-20% der Kunden gemäss Modell-Score. In diesem Bereich liegt der Lift bei über 3.5×, jeder kontaktierte Kunde hat eine mehr als dreifach höhere Kaufwahrscheinlichkeit als der Durchschnitt.

ROI-Beispielrechnung

Angenommen, eine E-Mail-Kampagne kostet CHF 0.50 pro Kontakt und ein Parfümkauf generiert CHF 80 Deckungsbeitrag. Bei 100'000 Kunden und 10% Kontaktbudget (10'000 E-Mails):

- Zufallsselektion: $10'000 \times 24\% \text{ Kauf Rate} = 2'400 \text{ Käufer} \rightarrow \text{CHF } 192'000 \text{ Ertrag}$
- Modellselektion: $10'000 \times (24\% \times 4.12 \text{ Lift}) \approx 9'888 \text{ Käufer} \rightarrow \text{CHF } 791'000 \text{ Ertrag}$
- Zusätzlicher Ertrag: ~CHF 600'000 bei gleichem Budgeteinsatz

Dieses vereinfachte Beispiel zeigt das erhebliche Potenzial modellbasierter Selektion. In der Praxis wären weitere Faktoren wie Response-Elastizität und Kannibalisierungseffekte zu berücksichtigen.

Die Feature-Importance-Analyse liefert Hinweise für die Datenerhebung: Warenkorbaktivität und Browsing-Verhalten sollten systematisch erfasst werden. Wenn technisch möglich, lohnt sich die Speicherung des Zeitpunkts letzter Käufe auf Kundenbasis, da Rezenz ein starker Prädiktor ist.

Die SHAP-Erklärungen können für individuelle Kundenansprache genutzt werden: Wenn ein Kunde hohe Scores vor allem wegen Warenkorbaktivität erhält, könnte eine "Warenkorb-Erinnerung" effektiver sein als eine generische Produktempfehlung.

8.2 Nächste Schritte für Produktivstellung

Vor einem produktiven Einsatz wären mehrere Validierungsschritte erforderlich. Zunächst sollte das Modell auf echten Kundendaten neu trainiert und evaluiert werden, um die Übertragbarkeit der Methodik zu prüfen. Anschliessend wäre ein kontrollierter A/B-Test sinnvoll, bei dem eine Kundengruppe modellbasiert selektiert wird und eine Kontrollgruppe zufällig, nur so lässt sich der tatsächliche Geschäftsmehrwert quantifizieren.

Für Real-Time-Scoring wäre zusätzliche Infrastruktur erforderlich: eine API, die Kundenfeatures entgegennimmt und Scores in Millisekunden ausgibt. Das vorliegende Modell ist für Batch-Scoring konzipiert.

8.3 Weiterführende Forschung

Aus wissenschaftlicher Sicht bieten sich mehrere Erweiterungen an. Uplift Modeling würde die Unterscheidung zwischen überzeugbaren und nicht-überzeugbaren Kunden ermöglichen und damit präzisere ROI-Schätzungen liefern. Zeitreihenmodelle könnten Saisonalität und individuelle Kaufzyklen berücksichtigen. Die Integration zusätzlicher Datenquellen, etwa Web-Session-Daten oder externe Konjunkturindikatoren, könnte die Vorhersagekraft weiter steigern.

Schliesslich wäre eine vertiefte Fairness-Analyse über weitere geschützte Merkmale (Geschlecht, Region) sowie eine Untersuchung intersektionaler Effekte (z.B. junge Frauen in bestimmten Regionen) für einen verantwortungsvollen KI-Einsatz relevant.

9. Referenzen

- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/2939672.2939785>
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*.
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*.
- Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A Survey on Bias and Fairness in Machine Learning. *ACM Computing Surveys*.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*.
- Rosset, S., Neumann, E., Eick, U., & Vatrik, N. (2001). Customer Lifetime Value Modeling and Its Use for Customer Retention Planning. *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Verbeke, W., Martens, D., & Baesens, B. (2014). Social Network Analytics for Churn Prediction in Telco: Model Building, Evaluation and Network Architecture. *Expert Systems with Applications*.
- XGBoost Developers. (2024). XGBoost Documentation. <https://xgboost.readthedocs.io/>

10. Anhänge

Die Pipeline wird mit ``make run-all`` ausgeführt. Alle Ergebnisse sind unter ``artifacts/`` und ``reports/`` zu finden.

Zentrale Code-Snippets

Datengenerierung (`src/data_prep.py`):

python

```
def make_data(n=800, seed=42):

    rng = np.random.default_rng(seed)

    age_groups = ["18-24", "25-34", "35-44", "45-54", "55+"]

    regions = ["DE", "CH", "FR", "IT", "AT"]

    df = pd.DataFrame({

        "views_7d": rng.poisson(3, size=n),

        "add_to_cart_30d": rng.poisson(1.2, size=n),

        "orders_12m": rng.poisson(0.6, size=n),

        "avg_price_viewed": rng.normal(65, 20, size=n).clip(10, 200),

        "brand_diversity": rng.integers(1, 8, size=n),

        "days_since_last_purchase": rng.integers(0, 365, size=n),

        "campaign_clicks": rng.poisson(0.8, size=n),

        "age_group": rng.choice(age_groups, size=n),

        "region": rng.choice(regions, size=n),

    })

    # Latenter Score mit Feature-Gewichtung

    score = (0.45 * df["views_7d"] + 0.9 * df["add_to_cart_30d"]

            + 0.8 * df["campaign_clicks"] + 0.6 * df["orders_12m"]

            - 0.003 * df["days_since_last_purchase"])

    # Logistische Transformation zur Wahrscheinlichkeit

    prob = 1 / (1 + np.exp(-(-2.0 + 0.25 * score)))

    df["bought_fragrance"] = (rng.random(n) < prob).astype(int)

    return df
```

Pipeline-Konstruktion (src/train.py):

python

```
def make_preprocessor(X: pd.DataFrame) -> ColumnTransformer:

    num_cols = X.select_dtypes(include=[np.number]).columns.tolist()

    cat_cols = [c for c in X.columns if c not in num_cols]

    return ColumnTransformer([

        ("num", StandardScaler(with_mean=False), num_cols),

        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),

    ])

# XGBoost mit automatischer Klassen-Gewichtung

clf = XGBClassifier(

    n_estimators=600, learning_rate=0.05, max_depth=5,

    subsample=0.9, colsample_bytree=0.9, reg_lambda=1.0,

    random_state=42

)

# scale_pos_weight automatisch berechnet: neg/pos ratio

model = Pipeline([("pre", pre), ("clf", clf)])
```

Lift-Berechnung (src/train.py):

python

```
def lift_at_k(y_true: np.ndarray, y_score: np.ndarray, k: float = 0.1):

    n = len(y_true)

    k_n = max(1, int(np.ceil(k * n)))

    order = np.argsort(-y_score)

    top_k = order[:k_n]

    precision_at_k = y_true[top_k].mean()

    baseline = y_true.mean()

    return float(precision_at_k / baseline)
```

KI-Offenlegung

https://github.com/kendricscoles/ml-future-fragrance/blob/main/docs/ki_offenlegung.md

Projekt Repository

<https://github.com/kendricscoles/ml-future-fragrance>